

Databricks Apps entwickeln

Werkzeuge, Konfigurationsdateien und wichtige Regeln für die Entwicklung von Apps mit Python oder Node.js.

Diese Seite beschreibt den typischen Entwicklungsablauf für Databricks Apps sowie die wichtigsten Dateien und Grenzen, die man dabei kennen sollte. Die Plattform übernimmt automatisch viele technische Details wie Workspace-URLs und Authentifizierung über Umgebungsvariablen, sodass sich Entwickler auf die eigentliche Anwendungslogik konzentrieren können.

Entwicklungsumgebung (IDE)

Man kann jede IDE (Entwicklungsumgebung) verwenden, die Python unterstützt, etwa PyCharm, IntelliJ IDEA oder Visual Studio Code. Databricks empfiehlt speziell die **Databricks-Erweiterung für VS Code**, es steht aber auch der eingebaute Notebook- und Datei-Editor innerhalb von Databricks zur Verfügung.

Typischer lokaler Entwicklungsablauf

1. App in der bevorzugten IDE bauen und testen
2. Lokal über die Kommandozeile ausführen und im Browser als Vorschau ansehen
3. Fertigen Code und Dateien in den Databricks-Workspace übertragen

Die zentrale Konfigurationsdatei: `app.yaml`

Die Datei `app.yaml` ist das Herzstück der Konfiguration einer App. Sie definiert:

- Einstellungen zur Laufzeitausführung (welcher Befehl startet die App)
- Individuelle App-Parameter
- Zugriff auf SQL-Warehouses
- Befehle und Argumente für die Ausführung der App

Wichtige Einschränkung: Dateigröße

App-Dateien dürfen **maximal 10 MB** groß sein. Überschreitet eine Datei im App-Verzeichnis diese Grenze, schlägt das Deployment mit einer Fehlermeldung fehl.

Zentrale Entwicklungsthemen

Die Dokumentation behandelt acht wesentliche Bereiche der App-Entwicklung:

1. **Apps-Umgebung** – verfügbare Programme, Umgebungsvariablen und Standard-Ressourcen (siehe eigene Seite)
2. **Definition der Laufzeitausführung** – Festlegen von Ausführungsbefehl, Argumenten und Standardwerten
3. **Compute-Konfiguration** – Auswahl von CPU und Arbeitsspeicher passend zum Workload
4. **Ressourcenverwaltung** – u. a. statische Dateien und Laufzeit-Assets
5. **Umgebungsvariablen** – für Authentifizierung, Workspace-Zugriff und app-spezifische Einstellungen
6. **Abhängigkeitsverwaltung** – Python-Abhängigkeiten über `pip` oder `uv`; Node.js-Abhängigkeiten über `npm`
7. **Token-Authentifizierung** – zur Anbindung von APIs während der lokalen Entwicklung
8. **Databricks AppKit** – ein Node.js/React-SDK mit typsicheren Datenbankabfragen und vorgefertigten UI-Komponenten

Automatische Umgebungskonfiguration

Die Plattform richtet automatisch mehrere Umgebungsvariablen ein, darunter Workspace-URLs und Authentifizierungsdaten. Das reduziert den manuellen Konfigurationsaufwand deutlich – man muss sich in der Regel nicht selbst um Verbindungsdetails zum Workspace kümmern.

Unterstützung nach Sprache

Sprache	Unterstützung
Python	Vollständig unterstützt über <code>pip</code> und <code>uv</code>
Node.js	Native Unterstützung mit <code>npm</code> -Abhängigkeitsverwaltung; zusätzlich beschleunigt das AppKit-SDK die Entwicklung

Quelle: <https://docs.databricks.com/aws/en/dev-tools/databricks-apps/app-development>

Stand: 2026-08-06